

Trabalho Prático — Modulação Digital em VHDL (Vivado)

Comunicação de Dados - Sistemas Reconfiguráveis

Objetivo

Desenvolver, simular e **implementar em FPGA**, utilizando o Vivado e a linguagem VHDL, sistemas de modulação digital do tipo **Unipolar NRZ-L** e **Unipolar NRZ-I**. O trabalho contempla a análise do comportamento dos sinais por meio de simulação no Vivado Simulator e a execução na placa física, com os *switches* representando a sequência de bits de entrada e os LEDs apresentando os pulsos modulados na saída.

Descrição da Atividade

Cada grupo deverá desenvolver **dois projetos independentes no Vivado**, sendo:

Projeto 1 — Modulação Unipolar NRZ-L

O projeto deverá conter:

- Um arquivo VHDL (**source**) com a implementação da modulação NRZ-L;
- Um arquivo VHDL (**testbench**) para simulação;
- Arquivo de **constraints** (.xdc) com o mapeamento dos pinos da placa.

Representação do sinal:

Na modulação Unipolar NRZ-L, o bit é representado pelo **nível do pulso durante todo o intervalo de bit**:

Bit	Pulso
1	nível alto
0	nível baixo

Projeto 2 — Modulação Unipolar NRZ-I

O projeto deverá conter:

- Um arquivo VHDL (**source**) com a implementação da modulação NRZ-I;
- Um arquivo VHDL (**testbench**) para simulação;
- Arquivo de **constraints** (.xdc) com o mapeamento dos pinos da placa.

Representação do sinal:

Na modulação Unipolar NRZ-I, o bit é representado pela **transição do pulso no início do intervalo de bit**:

Bit	Pulso
1	ocorre inversão do nível
0	mantém o nível anterior

Entrada de Dados

- A entrada de bits deverá ser um vetor com a **mesma quantidade de switches da FPGA** (ex: 16 bits).
- Cada *switch* da placa representa um bit da sequência a ser modulada.
- No testbench, essa sequência deverá ser aplicada ao longo do tempo de forma equivalente à entrada física.

Saída de Dados

- A saída modulada deverá ser apresentada nos **LEDs da placa**, refletindo o pulso correspondente a cada bit ao longo do tempo.
- Cada bit modulado deve permanecer ativo no LED durante todo o intervalo de bit configurado.

Duração do Pulso

A duração de cada bit (intervalo de símbolo) deverá ser **parametrizável** tanto no circuito principal quanto no testbench.

Circuito Principal

- A duração do pulso deve ser definida em função do **número de ciclos de clock**;
- Recomenda-se o uso de **generics** ou constantes;
- Exemplo: número de ciclos por bit.

A duração do pulso deve ser definida em função do número de ciclos de clock, não por assumir periodicidade do sinal, mas por se tratar de um sistema digital síncrono, no qual o tempo é discretizado pelo clock.

Para a execução na placa, o valor da contagem deve ser dimensionado de modo que o pulso seja **visualmente perceptível nos LEDs** (recomenda-se intervalos na ordem de centenas de milissegundos a alguns segundos por bit).

Testbench

- A duração do pulso deve ser definida em unidade de tempo;
- Deve ser facilmente modificável (ex: constante `BIT_PERIOD`);
- Recomenda-se valores na ordem de nanossegundos (ex: 20 ns);
- A sequência de bits deve respeitar esse intervalo.

Simulação

Os alunos deverão utilizar o **Vivado Simulator** para:

- Executar a simulação comportamental;
- Visualizar as formas de onda;
- Validar o funcionamento da modulação antes da gravação na FPGA.

Implementação na FPGA

Após a validação da simulação, os grupos deverão realizar a **síntese, implementação e gravação do bitstream** na placa FPGA. O sistema gravado deverá:

- Ler a sequência de bits configurada pelos *switches*;
- Iniciar a transmissão a partir de um sinal de *start* ou *reset* mapeado em um botão (*push-button*) da placa;
- Apresentar o pulso modulado nos LEDs correspondentes;
- Permitir a alteração da sequência de entrada via *switches* sem necessidade de regravação.

Atividades Extras (Opcionais com Pontuação Adicional)

Extra 1 — Saída pelo conector PMOD (Osciloscópio)

Disponibilizar o sinal modulado em um pino do conector **PMOD** da placa, de forma que possa ser observado em um **osciloscópio** físico.

- Adicionar a saída do sinal modulado a um pino do PMOD no arquivo de *constraints*;
- Capturar a forma de onda no osciloscópio para os dois tipos de modulação (NRZ-L e NRZ-I);
- Apresentar as capturas no relatório, identificando claramente os sinais.

Extra 2 — Saída VGA com cores pulsantes

Utilizar a **saída VGA** da placa para apresentar visualmente o pulso modulado, alterando a cor de tela cheia conforme o nível do sinal:

- Pulso em **nível baixo** (0): tela em uma cor (ex: preto ou azul);
- Pulso em **nível alto** (1): tela em outra cor (ex: branco ou verde);
- A tela deve "piscar" conforme a sequência modulada é transmitida.

Sugestão: implementar o controlador VGA padrão (640x480 @ 60Hz) e gerar os sinais RGB em função do bit modulado atual.

Entrega via GitHub

A entrega deste trabalho será realizada por meio de um **repositório no GitHub**, seguindo o fluxo *fork + pull request*. O conteúdo esperado em cada documento (READMEs, tutoriais, documentação técnica e relatório) está descrito em detalhes no **Guia de Documentação** disponibilizado junto a este enunciado.

Repositório oficial do trabalho:

`https://github.com/viniciussbrg/modulacao\_nrz\_l\_nrz\_i\_vhdl\_fpga`

Passo a passo da entrega

1. Fazer fork do repositório oficial:

- Acessar o repositório oficial (link acima);
- Clicar no botão **Fork** no canto superior direito da página;
- Na tela seguinte, manter o nome sugerido e clicar em **Create fork**;
- O GitHub criará uma cópia do repositório na conta do aluno, em `github.com/<aluno>/modulacao_nrz_l_nrz_i_vhdl_fpga`.

2. Clonar o fork para a máquina local (apenas um integrante por grupo precisa fazer isso):

```
git clone https://github.com/<aluno>/modulacao_nrz_l_nrz_i_vhdl_fpga.git
```

Substituir <aluno> pelo nome de usuário do GitHub.

3. **Desenvolver o trabalho** no fork do grupo, seguindo a estrutura de pastas descrita neste enunciado e o conteúdo descrito no Guia de Documentação;
4. **Enviar as alterações** (*commits* e *push*) para o fork ao longo do desenvolvimento:

```
git add .
git commit -m "Mensagem descritiva da alteração"
git push origin main
```

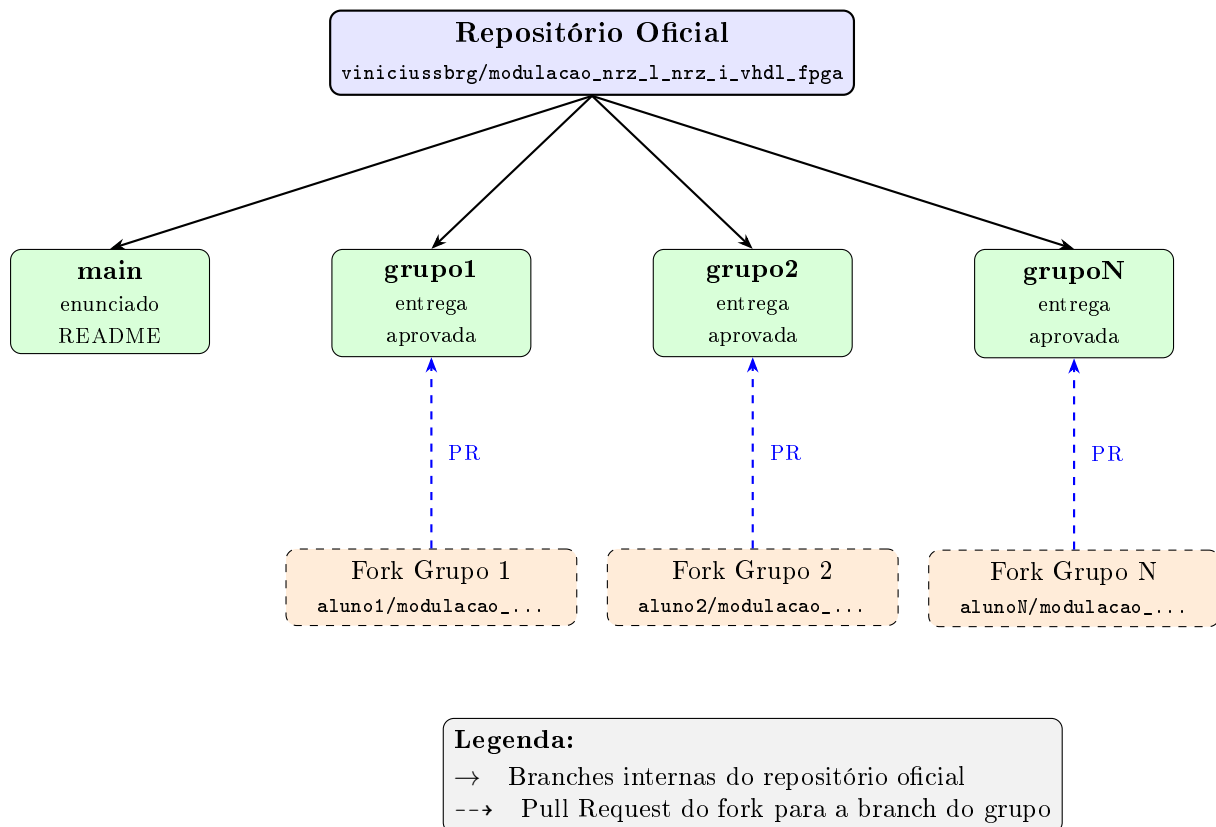
5. **Abrir um Pull Request** ao concluir o trabalho:

- No fork do aluno, clicar em **Contribute** → **Open pull request**;
- Verificar se o destino é o repositório oficial
(`viniciussbrg/modulacao_nrz_l_nrz_i_vhdl_fpga`);
- Definir o título no formato:
Entrega - Grupo X - Nome dos integrantes;
- Adicionar uma breve descrição com os nomes dos integrantes;
- Clicar em **Create pull request**.

6. **Aguardar a revisão do professor.** Após aprovação, o trabalho será incorporado ao repositório oficial em uma **branch dedicada do grupo**, preservando a autoria de todos os *commits*.

Organização do repositório oficial

A estrutura abaixo ilustra como o repositório oficial e os forks dos grupos se relacionam ao longo do trabalho:



Estrutura sugerida de pastas dentro do fork do grupo

Cada projeto (NRZ-L e NRZ-I) deve ser **auto-contido**: contendo seu próprio código, tutoriais, documentação e atividades extras. Dessa forma, alguém que entre na pasta do projeto consegue reproduzir todo o trabalho sem depender de arquivos externos.

```
modulacao_vhdl/
|-- README.md                (visao geral, integrantes, indice)
|
|-- nrz_l/
|   |-- README.md            (descricao e indice do projeto)
|   |-- src/
|   |   '-- nrz_l.vhd         (codigo-fonte)
|   |-- sim/
|   |   '-- nrz_l_tb.vhd      (testbench)
|   |-- constraints/
|   |   '-- nrz_l.xdc         (mapeamento de pinos)
|   |-- vivado_project/      (pasta gerada pelo Vivado)
|   |-- docs/
|   |   |-- tutorial_simulacao.pdf (passo a passo da simulacao)
|   |   |-- tutorial_placa.pdf    (passo a passo da gravacao na FPGA)
|   |   |-- documentacao_projeto.pdf (descricao tecnica e analise)
|   |   '-- midia/             (prints, fotos, videos)
|   '-- extras/               (opcionais deste projeto)
|       |-- pmod_osciloscopio/
|       |   |-- README.md      (descricao do extra)
|       |   |-- src/
|       |   |-- constraints/
|       |   '-- docs/
|       |       |-- tutorial_pmod.pdf
|       |       '-- midia/
|       '-- vga/
|           |-- README.md      (descricao do extra)
|           |-- src/
|           |-- constraints/
|           '-- docs/
|               |-- tutorial_vga.pdf
|               '-- midia/
|
|-- nrz_i/                    (mesma estrutura do nrz_l)
|   |-- README.md
|   |-- src/
|   |-- sim/
|   |-- constraints/
|   |-- vivado_project/
|   |-- docs/
|   '-- extras/
|
'-- relatorio_final/
```

-- relatorio.pdf	(documento consolidado do trabalho)
'-- video_demo.mp4	(ou link disponibilizado no README)

Conteúdo esperado da pasta docs/ de cada projeto

- `tutorial_simulacao.pdf` — passo a passo de como simular o circuito no Vivado: criar/abrir o projeto, adicionar arquivos-fonte, configurar e executar a simulação, interpretar o *waveform*. Deve permitir que outra pessoa reproduza a simulação do zero.
- `tutorial_placa.pdf` — passo a passo de como sintetizar, gerar o *bitstream*, conectar a placa e gravar o circuito. Deve conter o mapeamento dos *switches* e LEDs utilizados.
- `documentacao_projeto.pdf` — descrição técnica do projeto: diagrama de blocos, explicação do código VHDL, decisões de projeto e análise dos resultados obtidos.
- `midia/` — pasta com as mídias brutas (prints da simulação, fotos da placa, vídeos, capturas do osciloscópio) referenciadas pelos PDFs.

Observações Importantes

- Cada modulação deve ser feita em **projeto separado**;
- Cada projeto deve conter **um source, um testbench e um arquivo de constraints**;
- A implementação na placa é **obrigatória** nesta etapa;
- As atividades extras (PMOD e VGA) são **opcionais** e somam pontuação adicional;
- Não modificar nem realizar *commits* diretamente em branches de outros grupos no repositório oficial;
- O foco continua sendo a **análise dos sinais**: simulação, execução na placa e (opcionalmente) visualização no osciloscópio e VGA.